Session 13 & 14: Advanced Python & Web Programming

---

1. File Handling in Python

Python provides simple functions to handle file operations:

File Modes:

- "r" – Read (default)
- "w" – Write (overwrites)
- "a" – Append
- "r+" – Read and Write

Basic operations:

python

CopyEdit

Writing to a file with open("demo.txt", "w") as f: f.write("Hello World") # Reading from a file with open("demo.txt", "r") as f: data = f.read()

Important functions:

- read(), readline(), readlines()
- write(), writelines()
- seek(), tell()
- close() or use with (auto-close)

---

2. Directory Access Permissions

Linux files and directories have permission flags:

- **Read (r)** = 4
- **Write (w)** = 2
- **Execute (x)** = 1

File permissions can be viewed/changed:

- ls -l list with permissions
-

chmod 755 filename change permissions

- chown user:group filename change ownership
- Python example:

python

CopyEdit

import os os.chmod("demo.txt", 0o644) # rw-r--r--

---

3. Controls Socket (Networking)

Socket Programming:

Used for communication between devices.

python

CopyEdit

import socket # Server s = socket.socket() s.bind(("localhost", 9999)) s.listen() conn, addr = s.accept() conn.send(b"Hello client") conn.close() # Client c = socket.socket() c.connect(("localhost", 9999)) print(c.recv(1024))

- **socket.AF_INET** → IPv4
- **socket.SOCK_STREAM** → TCP
- **socket.SOCK_DGRAM** → UDP

---

4. Libraries and Functional Programming

- Python has extensive standard and third-party libraries.
- Use import to include modules.
- Example: import math, import os, import requests

Functional Programming Concepts:

- **First-class functions**: functions passed like variables
- **Lambda**: anonymous functions
- **map(), filter(), reduce()**:

python

CopyEdit

from functools import reduce nums = [1, 2, 3, 4] squares = list(map(lambda x: x**2, nums)) evens = list(filter(lambda x: x % 2 == 0, nums)) total = reduce(lambda x, y: x + y, nums)

---

5. Server and Client Architecture

- **Client-Server Model**: Clients send requests; servers process and respond.
- **Server**: Central system that provides services.
- **Client**: Requests resources from the server.
- Examples:

- Web Server (HTTP)
- Database Server (MySQL, PostgreSQL)
- Email Server (SMTP, POP3)

🌐 6. Web Servers and Client Scripting

Web Server:

- Responds to HTTP requests.
- Examples: Apache, Nginx, Python’s http.server

bash

CopyEdit

```
# Start simple server python3 -m http.server 8000
```

Client-Side Scripting:

- Runs in browsers (e.g., JavaScript)
- Enhances interactivity
- Communicates with server via AJAX, fetch API

🌐 7. Introduction to Python Web Development Frameworks

Flask:

- Lightweight Python web framework
- Easy for beginners and prototyping

python

CopyEdit

from flask import Flask app = Flask(__name__) @app.route("/") def hello(): return "Hello from Flask!" app.run()

Django:

- Full-fledged web framework

- Includes ORM, templating, admin, etc.

Others: FastAPI, Tornado, Pyramid